ChatGPT

Adding Amazon EKS to an AWS Infrastructure Decision Utility

Executive summary

Amazon EKS should be added as a **supported but gated** compute option in your decision utility, not as a default recommendation. Official AWS documentation positions EKS as managed Kubernetes for workloads that need Kubernetes APIs, ecosystem tooling, or portability across standard Kubernetes environments, while ECS is the lower-ops AWS-native container orchestrator, AWS Fargate is the serverless container execution layer, and Lambda is the serverless function option with a maximum 900-second execution time. As of April 19, 2026, standard EKS clusters also introduce a non-trivial fixed control-plane fee of **$0.10 per cluster-hour**, rising to **$0.60 per cluster-hour** in extended support, whereas ECS has no separate control-plane charge and Fargate/Lambda are primarily usage-priced. [1]

For a rules engine, the key conclusion is this: **choose EKS when the user is blocked by orchestration semantics, platform standardization, custom networking, multi-tenancy, or specialized stateful/container platform requirements**; otherwise bias toward ECS/Fargate or Lambda. In practical product terms, EKS is most justified when the user needs one or more of the following: Kubernetes-native extensibility such as Helm/operators/controllers, custom CNI behavior or richer network segmentation, shared multi-tenant clusters with namespace/RBAC/quota policy, portable Kubernetes manifests across environments, or stateful workloads that need CSI-backed storage classes beyond what Fargate can support. [2]

The design implication for Terraform generation is equally important: **Terraform should remain the final rendering step of a normalized architecture spec**, not the center of the product. The utility should first determine whether EKS is warranted, then collect the additional decisions EKS requires—cluster access model, network exposure, compute mix, storage classes, workload identity, ingress strategy, autoscaling strategy, and observability—and only then emit Terraform modules in a predictable dependency order. HashiCorp's own guidance supports splitting modules along **privilege boundaries** and **volatility**, keeping provider configurations in the root module, and committing the `.terraform.lock.hcl` lock file for reproducibility. [3]

Scope and unspecified assumptions

This report evaluates **standard Amazon EKS clusters** as the omitted compute option in an AWS infrastructure decision utility. It includes managed node groups, optional Fargate profiles, add-ons, ingress, identity, storage, and Terraform module structure. EKS Auto Mode is mentioned only where it materially changes a recommendation, because your requested module/resource model centers on standard EKS clusters with explicit decisions around node groups, Fargate profiles, CNIs, IRSA/Pod Identity, and add-ons. [4]

The following inputs are **unspecified** in your prompt and therefore should remain explicit variables in the utility rather than hidden assumptions:

Assumption area	Status	Why it matters
Target customer size	Unspecified	Small teams with few services should be biased away from EKS because EKS adds control-plane cost and platform overhead; larger platform teams can amortize that overhead. [5]
Single-account vs multi-account	Unspecified	AWS recommends multi-account strategies for isolation, quota scaling, and simpler IAM boundaries; that materially changes Route53, VPC endpoint, IAM, and Terraform layout. [6]
Single-region vs multi-region	Unspecified	Drives whether Route53 failover, CloudFront, cross-region ECR, and data topology must be modeled. [7]
Public internet exposure model	Unspecified	Determines ALB/NLB, API Gateway, CloudFront, public/private cluster endpoint settings, and subnet layout. [8]
Compliance regime	Unspecified	Determines whether to require private cluster, CMKs, control-plane logs, stricter Pod Security Standards, Bottlerocket/CIS validation, and image scanning. [9]
Runtime maturity of the customer team	Unspecified	EKS requires more customer responsibility for security “in” the cluster than Lambda/Fargate and more platform skill than ECS. [10]

One terminology note is important for the comparison that follows: **Fargate is not an orchestrator**. It is a serverless compute engine that can run under ECS or under EKS. It still deserves a separate row in your utility because users frequently need to decide whether they want **serverless pods/tasks** versus **node-based container capacity**, even after they have picked an orchestrator. [11]

Decision criteria for choosing EKS

EKS is favored when the decision problem is really about **platform capability** rather than “how do I run one container.” AWS describes EKS as managed Kubernetes, and AWS’s CLI reference emphasizes that applications running on EKS are fully compatible with standard Kubernetes environments. That materially lowers application-level lock-in relative to ECS, especially for teams that already rely on Helm charts, operators, Kubernetes schedulers, or policies that assume upstream Kubernetes objects and behavior. [12]

By contrast, ECS is favored when the user wants container orchestration without Kubernetes control-plane concepts. AWS describes ECS as a fully managed container orchestrator with AWS operational best practices built in and no control-plane complexity to manage. Fargate further lowers operational burden by eliminating server management and charging for requested vCPU, memory, and storage usage from image pull until task or pod termination. Lambda lowers operations even more when the workload can be expressed as event-driven functions rather than long-lived services. [13]

A useful gating principle for your utility is therefore: **if the user’s blockers are “what compute should run my code?” use Lambda/ECS/Fargate first; if the blockers are “what container platform semantics do I**

need?" then evaluate EKS. That is where EKS becomes valuable—custom networking, tenant isolation, stateful orchestration, mixed compute types, and portability—not merely because the user can type Terraform for a cluster. [14]

Concise comparison table

The table below is a synthesis of official AWS service docs, pricing pages, and container decision guidance. "Fargate" is shown separately because your utility needs to decide serverless-container capacity independently of ECS/EKS control planes. [15]

Option	Best workload fit	Ops burden	Cost profile	Scaling model	Portability	Networking complexity
Lambda	Event-driven, asynchronous, request/response functions, short-lived jobs	Lowest	Pure usage-based by requests and duration; no idle infrastructure	Per-invocation concurrency; no nodes or pods	Lowest to moderate	Lowest
Fargate	Stateless or lightly stateful containers where users want containers without node management	Low	Usage-based on requested vCPU, memory, storage; no idle node waste, but no node-density optimization	Per task or pod	Moderate	Medium
ECS	AWS-native container services where Kubernetes is not required	Low to medium	No ECS control-plane fee; pay for EC2/managed capacity or Fargate	Service/task scaling	Low to moderate	Medium
EKS	Kubernetes-native platforms, multi-tenant clusters, custom networking/policy, richer stateful orchestration, portability	Highest	Per-cluster fee plus compute/storage/network; extended support can add materially more	HPA/VPA patterns plus Cluster Autoscaler or Karpenter and Kubernetes scheduling semantics	Highest	Highest

Practical decision rules for an EKS branch

The most reliable way to add EKS to your utility is to treat it as a **conditional branch with hard triggers and hard disqualifiers**.

User intent signal	Bias	Reasoning
Needs Helm charts, operators, CRDs, admission control, or Kubernetes-native policy engines	**Bias toward EKS**	These are Kubernetes platform semantics; ECS/Fargate do not substitute for them. EKS runs standard Kubernetes workloads and tooling. [16]
Needs shared multi-tenant clusters with namespaces, RBAC, quotas, and network policy per tenant	**Bias toward EKS**	AWS's EKS tenant-isolation guidance explicitly uses namespaces, roles, role bindings, quotas, and network policies for soft multi-tenancy. [17]
Needs DaemonSets, privileged containers, `HostPort`, `HostNetwork`, or EBS-backed pod volumes	**Bias toward EKS on EC2 nodes**	EKS Fargate does not support DaemonSets, privileged containers, `HostPort`, `HostNetwork`, or EBS volumes for pods. [18]
Needs custom eBPF networking/ observability and may want Cilium	**Bias toward EKS on EC2 nodes**	EKS Fargate cannot use alternate CNI plugins; Cilium chaining with AWS VPC CNI is possible on EKS, but some advanced Cilium features are limited when chained. [19]
Needs simple web/API containers, AWS-native operations, and team has low Kubernetes maturity	**Bias toward ECS or ECS+Fargate**	ECS is fully managed, integrates deeply with AWS, and avoids Kubernetes control-plane and policy complexity. [20]
Needs highly bursty event-driven execution and work completes in 15 minutes or less	**Bias toward Lambda**	Lambda is usage-based and caps execution at 900 seconds. [21]
Strong portability or anti-lock-in requirement across standard Kubernetes environments	**Bias toward EKS**	AWS states EKS applications are compatible with standard Kubernetes environments. [22]
Very small workload footprint with strong pressure to avoid fixed platform cost	**Bias away from EKS**	EKS introduces a control-plane fee even before worker cost; ECS has no separate control-plane fee and Lambda/Fargate are pure usage-priced. [23]

A strong product rule is to require **at least one "hard EKS trigger"** before exposing the EKS path as a recommended option. If no hard trigger is present, the utility should explain why ECS/Fargate or Lambda is likely the safer and cheaper path. That will reduce false-positive EKS recommendations and keep Terraform output aligned with actual platform need rather than platform fashion. [24]

Required AWS resources and typical module boundaries for EKS

At the infrastructure level, an EKS design utility must reason about more than “cluster + nodes.” AWS’s EKS networking guidance requires a VPC with sufficient IPs, and EKS expects at least two subnets for cluster creation. For private clusters without outbound internet, AWS requires private endpoint access and often a wider set of interface or gateway endpoints for ECR, S3, STS or EKS Auth, EC2, ELB, CloudWatch Logs, and EKS APIs. [25]

The diagram below shows the dependency shape your utility should model before Terraform generation. It intentionally separates **foundation**, **cluster control plane**, **compute**, **add-ons**, **edge**, and **data** because those boundaries line up well with Terraform composition and with the operational privilege/volatility split HashiCorp recommends. [26]

```
flowchart TD
    U[User intent spec] --> N[Network foundation]
    U --> S[Security baseline]
    U --> C[EKS control plane]
    N --> C
    S --> C

    C --> I[Identity and access]
    C --> A[Core add-ons]
    C --> X[Observability]
    C --> G[Ingress and edge]
    C --> D[Storage and data access]

    I --> P1[Pod Identity or IRSA]
    C --> MNG[Managed node groups]
    C --> FP[Fargate profiles]
    C --> KA[Karpenter or Cluster Autoscaler]

    A --> VPC[VPC CNI or Cilium]
    A --> KP[kube-proxy]
    A --> CD[CoreDNS]
    A --> CSI[EBS EFS FSx CSI add-ons]

    G --> ALB[AWS Load Balancer Controller]
    G --> NLB[NLB services]
    G --> APIGW[API Gateway VPC Link]
    G --> CF[CloudFront]
    G --> R53[Route53]

    D --> ECR[ECR]
    D --> EBS[EBS]
    D --> EFS[EFS]
```

```
    D --> FSX[FSx]
    D --> DB[RDS DynamoDB]
```

Resource list for an EKS cluster path

The matrix below consolidates the core AWS and Kubernetes resources that recur across official EKS docs for networking, private clusters, add-ons, storage, and ingress. Rows marked “conditional” should be emitted only if the user’s answers make them necessary. [27]

Resource area	Typical resources	Required or conditional	Key user inputs	Why it exists
Network foundation	VPC, at least two subnets, route tables, NAT and/or VPC endpoints, security groups	Required	CIDRs, AZ count, egress model, public ingress needed	EKS requires VPC/subnet capacity; private clusters need endpoint planning. [28]
EKS control plane	`aws_eks_cluster`, cluster IAM role, cluster security group, endpoint settings	Required	Cluster name, Kubernetes version, public/private endpoint mode, log types	Core managed control plane. [29]
Human cluster access	EKS access entries/CAM, RBAC bindings	Required	Admin roles, readonly roles, CI roles, Identity Center/OIDC choice	AWS recommends CAM over new `aws-auth` usage for new clusters. [30]
OIDC provider	IAM OIDC provider for the cluster	Conditional but effectively required for IRSA or add-ons that need it	IRSA enabled, add-ons needing IAM	IRSA requires a cluster OIDC provider; many add-ons needing IAM assume it. [31]
Workload identity	EKS Pod Identity agent and associations, or IRSA roles/policies	Required for AWS-integrated workloads	Pod Identity vs IRSA, namespaces/service accounts, cross-account access	Pod Identity is preferred for EC2-node clusters; IRSA remains necessary in cases Pod Identity cannot cover. [32]

Resource area	Typical resources	Required or conditional	Key user inputs	Why it exists
Compute on EC2	Managed node groups, node IAM role, launch template, taints/labels	Conditional	Instance families, arch, GPU, min/max, Spot allowed, AZ/topology	Default path for general EKS workloads, especially when DaemonSets, EBS, or custom CNIs are needed. [33]
Compute on Fargate	Fargate profiles, pod execution role, subnet selectors	Conditional	Namespaces/labels, private subnets, profile boundaries	Good for isolated, low-ops pods; requires private subnets and a pod execution role. [34]
Core networking add-ons	VPC CNI, CoreDNS, kube-proxy	Required	CNI mode, versions, network-policy enabled	Standard EKS networking and core cluster services. [35]
Optional alternate policy/network stack	Cilium in AWS VPC CNI chaining mode or full Cilium migration	Conditional	Need for eBPF observability/encryption/policy	Chaining is supported but some advanced Cilium features are limited. [36]
Autoscaling	Cluster Autoscaler or Karpenter, HPA support	Usually required in production	Autoscaler choice, Spot, heterogeneous node pools, burst patterns	Karpenter improves right-sizing and reduces node-group sprawl; CA remains viable for ASG-based scaling. [37]
Registry	ECR repositories, repository policies, lifecycle policy, scanning config	Usually required	Repository names, tag policy, retention, scanning level	ECR provides repository controls, lifecycle, and vulnerability scanning. [38]

Resource area	Typical resources	Required or conditional	Key user inputs	Why it exists
Ingress and load balancing	AWS Load Balancer Controller, ALB/NLB, optional API Gateway VPC link	Conditional	HTTP vs TCP, internal vs internet-facing, WAF/CDN/API needs	ALB/NLB exposure is controller-driven on EKS; API Gateway can privately integrate to ALB/NLB using VPC links. [39]
Observability	CloudWatch Observability add-on / Container Insights, control-plane logs, log groups	Strongly recommended	Enhanced observability, App Signals, log retention	CloudWatch add-on installs CloudWatch Agent and Fluent Bit; control-plane logs are separate cluster settings. [40]
Storage	EBS CSI, EFS CSI, FSx CSI, storage classes, KMS keys where needed	Conditional	Block vs shared filesystem vs HPC vs ONTAP/ZFS	Statefulness on EKS is a first-class decision, not an afterthought. [41]

Compute, CNI, and storage choices the rules engine should expose

For **EC2-backed EKS**, the default compute question should be whether the workload wants **managed node groups only** or **managed node groups plus Karpenter**. Karpenter is the better default for heterogeneous or bursty clusters because AWS describes it as cluster-aware, right-sizing compute based on real pod requirements and avoiding the need to pre-build many node groups. Cluster Autoscaler is still appropriate when the customer explicitly wants ASG-driven scaling and simpler node-group mental models. [42]

For **Fargate-backed pods**, the utility should ask a separate gating question: “Can this workload tolerate the Fargate limitations?” The official EKS Fargate docs state that Fargate does **not** support DaemonSets, privileged containers, `HostPort`, `HostNetwork`, or alternate CNIs; pods run in private subnets; and EBS-backed pod volumes are not supported. If the answer is no, the utility should automatically switch back to EC2-backed node groups. [43]

For **networking**, `aws-vpc-cni` should remain the default recommendation because it is the native, AWS-supported EKS networking path and gives pods VPC-native IP addresses. Expose **Cilium** as an advanced option only when the user explicitly needs richer eBPF features, enhanced policy or observability, and accepts the additional operational burden that comes with chaining or migration. In chaining mode, Cilium

itself documents that some advanced features, including Layer 7 policy and IPsec transparent encryption, may be limited. [44]

For **storage**, the utility should ask **block vs shared filesystem vs high-performance filesystem vs portable self-managed storage**. EBS CSI is the default for zonal block volumes and most Kubernetes stateful patterns, but it cannot mount to Fargate pods. EFS is the default shared POSIX filesystem and can be mounted from Fargate pods, although Fargate supports only static provisioning for EFS volumes. FSx for Lustre is the AWS-native HPC/ML filesystem path, while FSx for NetApp ONTAP and FSx for OpenZFS fit enterprise file semantics and ZFS-like workflows. Rook/Ceph should be treated as an advanced, portability-oriented choice because production-grade replicated or erasure-coded Ceph requires at least three worker nodes and significantly more storage operations expertise. [45]

Recommended Terraform module boundaries

HashiCorp recommends grouping infrastructure by **encapsulation**, **privilege boundaries**, and **volatility**, and keeping provider configurations in the root module while child modules declare provider requirements. That maps especially well to EKS because network, IAM, and security controls are high-privilege and low-volatility, while node groups, add-ons, and app-facing edge modules are higher-churn. [26]

Suggested module	Owns	Why this boundary works
`foundation-network`	VPC, subnets, route tables, NAT, VPC endpoints, baseline SGs	High privilege, low volatility, shared across clusters/ environments
`foundation-kms`	CMKs for EKS, EBS, logs if customer-managed keys are required	Security-critical and separately governed
`eks-cluster`	EKS cluster, endpoint mode, control-plane logs, access entries, OIDC	Core control-plane boundary
`eks-nodegroups`	Managed node groups, launch templates, node roles, labels/taints	Frequent changes in size, instance families, Spot mix
`eks-fargate-profiles`	Fargate profiles and pod execution role	Independent compute branch with different constraints
`eks-addons-core`	VPC CNI, CoreDNS, kube-proxy, metrics prerequisites	Core cluster-operability surface
`eks-addons-platform`	Pod Identity agent, CloudWatch add-on, EBS/EFS/FSx CSI, ALB Controller, Karpenter or CA	Optional capabilities that vary by customer intent
`registry-ecr`	ECR repos, lifecycle, scanning, replication if needed	Clear ownership and separate retention policy concerns
`edge-routing`	ALB/NLB resources, Route53, CloudFront, API Gateway VPC link	Internet exposure and DNS lifecycle differ from cluster lifecycle

Suggested module	Owns	Why this boundary works
`data-access`	Storage classes, CSI config, DB SG rules, gateway endpoints	Application state and service connectivity policy boundary
`observability`	Log groups, alarms, dashboards, retention, Container Insights settings	Platform telemetry changes independently of network or cluster creation

Security and compliance guardrails

Security is one of the strongest reasons to avoid a simplistic “generate cluster.tf” product shape. AWS’s EKS security guidance explicitly frames EKS as a shared-responsibility model: AWS manages the managed control plane, but the customer remains largely responsible for IAM, pod security, runtime security, and network security inside the cluster. In other words, supporting EKS in your utility requires encoding **opinionated guardrails**, not just resources. [46]

A sensible default for **human and CI access** is to prefer **Cluster Access Management (CAM) / access entries** over the legacy `aws-auth` ConfigMap. AWS’s best-practices guidance says ConfigMap-based access management is deprecated in favor of CAM for new clusters. Your utility should therefore ask for “cluster admins,” “platform operators,” “read-only users,” and “CI principals,” then emit both access-entry configuration and the corresponding Kubernetes RBAC groups. [30]

For **workload identity**, the preferred modern path is **EKS Pod Identity** for EC2-backed EKS nodes, because AWS documents benefits such as least privilege, credential isolation, reusability, and lower STS load per pod. However, Pod Identity is available only on Linux EC2 worker nodes and is not available on AWS Outposts or EKS Anywhere, so your utility must still support **IRSA**. In practice, that means: default to Pod Identity when compute includes EC2 Linux nodes; fall back to IRSA when the workload runs on Fargate or in environments Pod Identity does not support. [47]

For **pod hardening**, the safest default is to enforce Kubernetes **Pod Security Standards** at least at **Baseline** for system namespaces and **Restricted** for application namespaces unless the user explicitly asks for exceptions. Kubernetes documents Baseline, Restricted, and Privileged as the standard policy tiers, and EKS security guidance places pod security squarely within customer responsibility. [48]

For **network isolation**, treat “all pods talk to all pods” as a failing default. AWS’s EKS network-security guide states that pod-to-pod communication is allowed by default and that this is not considered secure. The utility should therefore default to **namespace-scoped default-deny network policies** for application namespaces, optionally with cluster-wide admin policies where supported. If using the native VPC CNI network-policy feature, your rules engine must also know that those policies apply to **EC2 Linux nodes only**, not to Fargate or Windows nodes. [49]

For **private clusters**, the utility should expose a first-class “private control plane / limited internet” mode. In that mode, the cluster should enable private endpoint access and, when there is no outbound internet, provision the expected VPC endpoints for ECR API, ECR DKR, S3, CloudWatch Logs, STS or EKS Auth depending on IRSA vs Pod Identity, ELB, EC2, and EKS. AWS also documents an important caveat: Route53

does **not** support AWS PrivateLink for this purpose, which affects fully private `external-dns` patterns. [50]

For **control-plane auditability**, enable all five EKS control-plane log types by default in production: `api`, `audit`, `authenticator`, `controllerManager`, and `scheduler`. AWS sends these logs to CloudWatch Logs and explicitly describes them as audit and diagnostic logs. This is one of the clearest "make it a default in the generator" decisions you can encode. [51]

For **encryption**, your minimum rule should be: rely on EKS's default API-data envelope encryption on Kubernetes 1.28+ and optionally require a customer-managed KMS key when the user selects stricter compliance. AWS states that EKS 1.28+ provides default envelope encryption for all Kubernetes API data via KMS v2, and EBS/EFS/other storage classes can then layer storage-specific encryption settings on top. [52]

For **image supply chain controls**, require ECR lifecycle policies and at least basic scanning, and recommend enhanced scanning where the customer wants continuous vulnerability visibility. AWS documents ECR lifecycle rules for automatic expiration management, and ECR enhanced scanning integrates with Amazon Inspector for continuous scanning of OS and language-package vulnerabilities. [53]

For **add-on lifecycle**, prefer AWS-managed EKS add-ons where AWS offers them—at minimum VPC CNI, CoreDNS, kube-proxy, and the relevant CSI drivers—so version compatibility and day-2 maintenance are easier to encode in Terraform. AWS's add-ons catalog explicitly lists these as managed add-ons, along with the Pod Identity Agent and the CloudWatch Observability agent. [54]

For **compliance and vulnerability analysis**, your utility should expose optional "compliance profile" toggles that turn on control-plane logs, Inspector/GuardDuty integrations, Bottlerocket preferences, and CIS validation hooks. AWS specifically calls out CIS EKS benchmark support, platform versions, Amazon Inspector, and GuardDuty in its vulnerability-analysis guidance, and Bottlerocket is documented as helpful for CIS and PCI-oriented hardening objectives. [55]

For **EKS Anywhere and Outposts**, the rule is simple: treat them as separate deployment families, not mere flags on the standard EKS branch. EKS on Outposts supports local or extended clusters with different availability and network assumptions, while EKS Anywhere is explicitly user-managed and places cluster lifecycle operations—upgrades, patching, and scaling—on the customer. That means the Terraform and operational defaults in this report apply most directly to EKS in AWS Regions, not to EKS Anywhere. [56]

Cost and operational tradeoffs

The single most important cost fact to build into the rules engine is that **EKS has a fixed cluster cost floor**. As of April 19, 2026, standard Kubernetes-version support on EKS costs **$0.10 per cluster-hour**, and extended support costs **$0.60 per cluster-hour**. AWS also documents that Kubernetes versions receive **14 months of standard support** and **12 months of extended support**, with extended support enabled by default unless explicitly changed. That creates a real ongoing platform charge and a real "upgrade discipline" requirement that ECS, Fargate, and Lambda do not impose in the same way. [57]

By contrast, ECS's pricing model has **no separate ECS control-plane fee**; customers pay for the underlying resources they run. Fargate charges for requested vCPU, memory, operating system, CPU architecture, and

storage from image pull until task or pod termination. Lambda charges by request and duration. This creates a strong design implication for your utility: for smaller or simpler services, EKS's fixed platform cost and version-management burden should count against it unless there is a compensating Kubernetes-specific requirement. [58]

The utility should also model **operations cost**, not just AWS bill cost. EKS brings node lifecycle, add-on lifecycle, Kubernetes version lifecycle, policy management, and workload scheduling considerations that Lambda and ECS hide or simplify. AWS's own EKS security guidance explicitly says customer responsibility is larger in EKS than in Fargate, and ECS is documented as avoiding control-plane complexity for container operations. [59]

For EC2-backed EKS, the main cost levers are **node purchase option**, **cluster density**, and **autoscaler choice**. AWS recommends both Cluster Autoscaler and Karpenter as node autoscalers, but Karpenter is especially useful when the customer wants just-in-time right-sizing and fewer pre-sized node groups. AWS also documents that larger nodes can improve usable capacity percentage by reducing per-node overhead from DaemonSets and reserves, though over-packing creates saturation risk. That means your utility should treat "steady high utilization + strong platform team" as a signal that EKS on EC2 can outperform serverless-container economics, while still warning about node-saturation telemetry. [60]

For **Spot and mixed-capacity strategies**, EKS should usually default to a **small On-Demand baseline plus Spot expansion pools** for fault-tolerant workloads. AWS documents that managed node groups can use Spot instances and that workloads can be scheduled to Spot or On-Demand groups according to fault tolerance. Karpenter can further improve cost efficiency by provisioning nodes that more precisely match actual pod requirements. [61]

For **Fargate on EKS**, the cost tradeoff is different: you eliminate node management and idle node waste, but you also give up important Kubernetes capabilities such as DaemonSets, alternate CNI, EBS-backed pod volumes, and various "node-level" platform patterns. A practical product rule is to use EKS Fargate only when the customer already needs Kubernetes semantics **and** wants a low-ops path for a subset of namespaces, not as the universal EKS default. [62]

```
xychart-beta
    title "Qualitative platform cost floor"
    x-axis ["Lambda","Fargate","ECS","EKS"]
    y-axis "Relative fixed platform cost floor" 0 --> 4
    bar [0,0,0,4]
```

The chart above is intentionally **qualitative**: Lambda, Fargate, and ECS do not impose an EKS-like per-cluster control-plane charge floor, while EKS does. It does **not** mean EKS is always more expensive end-to-end; at sustained scale, EC2-backed EKS can offset that fixed fee through node density, Spot, and platform standardization. It means the utility must model **baseline platform cost** separately from **workload execution cost**. [63]

Integration patterns with other AWS services

For **ingress**, the default AWS-native pattern on EKS is the **AWS Load Balancer Controller**. AWS documents that the controller watches Kubernetes `Ingress` and `Service` resources and provisions the appropriate AWS Elastic Load Balancing resources. In your utility, this should be the default answer when the user wants Kubernetes-native L7 exposure with ALB or service-backed NLB exposure from the cluster. [64]

For **private APIs or front-door separation**, expose **API Gateway + VPC Link** as a distinct pattern. AWS documents that VPC links V2 support private integrations to ALB, NLB, or Cloud Map resources, and that both HTTP APIs and REST APIs can use VPC links for private integrations. At the same time, AWS's REST-vs-HTTP comparison notes that only REST APIs have the **Private endpoint type**, while HTTP APIs do not. This means your utility should separately ask: "Do you need API Gateway features?" and "Do you need a private API endpoint type or merely a private integration to your EKS backend?" [65]

For **CDN and edge acceleration**, AWS documents that CloudFront can use ALB, NLB, API Gateway, EC2, and S3 as origins. That makes CloudFront an appropriate edge layer for EKS-hosted applications when the user wants caching, global POP distribution, TLS termination control, or a protected front door in front of ALB. Route53 alias records then provide the normal DNS path to CloudFront or directly to ELB resources. [66]

For **database access**, a common secure pattern is private networking and security-group segmentation. AWS's EKS guidance on security groups for pods explicitly points out that allowing RDS inbound only from node security groups can be too coarse and that security groups for pods improve segmentation. Your utility should therefore ask whether the user needs **pod-level** versus **node-level** database access controls. [67]

For **DynamoDB and S3 access**, the utility should prefer **gateway endpoints** where possible in private VPC designs. AWS documents that S3 and DynamoDB gateway endpoints avoid the need for an internet gateway or NAT device and are available at **no additional cost**. This is especially important for EKS private-cluster profiles, because it lowers both cost and attack surface. [68]

For **private-cluster AWS service access**, your integration model should explicitly add interface endpoints or gateway endpoints for the AWS services the cluster and pods need. AWS's private-cluster guidance lists ECR, ELB, CloudWatch Logs, EC2, STS, EKS Auth, and EKS among the common endpoints. A useful rule is: if the user chooses private cluster with no outbound internet, the utility must expand into an **endpoint planning** branch rather than pretending the cluster is just a VPC flag. [69]

One subtle but important private-cluster integration caveat is that AWS documents a limitation for `external-dns`**-style Route53 management** from a fully private cluster, because Route53 does not support AWS PrivateLink for that path. If your utility generates a private-cluster architecture and the user asks for in-cluster DNS automation, it should either warn about this limitation or steer DNS writes to a separate network path or automation tier. [70]

Terraform generation guidance

The Terraform output for EKS should be driven by a **normalized intermediate spec** and rendered through modules that mirror the boundaries described earlier. HashiCorp's module guidance is directly applicable

here: keep provider configurations in the **root module**, let child modules declare provider requirements, and split code according to privilege and volatility. HashiCorp also documents that `.terraform.lock.hcl` belongs to the whole configuration, is created in the root working directory, and should be committed to version control. For AWS backends, the Terraform S3 backend supports `use_lockfile = true` for state locking. [71]

A practical root-module pattern for your generator is:

- **Root module**: backend, providers, environment/global locals, tags, module calls, cross-module wiring.
- **Reusable child modules**: network, cluster, node groups, Fargate profiles, add-ons, ingress, observability, registry, storage.
- **Environment specs**: JSON or YAML documents capturing user intent and resolved architecture choices.
- **Module manifest**: ordered list of module invocations and dependency outputs.

That shape keeps the decision utility focused on architecture resolution while making the Terraform layer deterministic and composable. [72]

Inputs the utility should require before generating EKS Terraform

Input group	Minimum fields to collect	Why it is required
Cluster basics	cluster name, region, environment, Kubernetes version, support policy	EKS pricing and lifecycle depend on version/support tier, and upgrades are operationally significant. [57]
Account topology	single-account or multi-account, shared-services account, DNS/account boundaries	Changes IAM, DNS, registry, and VPC endpoint architecture. [6]
Network	VPC CIDR, AZ count, public/private subnet strategy, NAT vs endpoints, private cluster yes/no	EKS networking and private-cluster endpoints are foundational. [73]
Compute	EC2 node groups and/or Fargate profiles, instance families, Spot allowed, GPU needed, ARM/x86	Compute path changes resource requirements and feature support. [74]
Autoscaling	Karpenter vs Cluster Autoscaler, min/max policies, disruption requirements	Determines node-scaling resources and cost posture. [75]
Identity	CAM users/roles, Pod Identity vs IRSA, service-account mappings	Required for both human and workload AWS access. [76]
Storage	EBS, EFS, FSx, Ceph/Rook, backup/snapshot requirements	Stateful workloads need explicit storage choices. [77]
Edge	ALB vs NLB vs API Gateway, CloudFront, Route53 zone ownership	Drives ingress and DNS modules. [78]

Input group	Minimum fields to collect	Why it is required
Security/ compliance	private cluster, CMK required, log retention, image scanning, Pod Security level	Required to emit secure-by-default infrastructure. [79]
Observability	CloudWatch add-on, Container Insights, App Signals, log destinations	Changes add-on and IAM wiring. [80]

Recommended intermediate spec

The schema below is a recommended **normalized architecture contract** between your decision engine and your Terraform renderer. Its fields are derived from the EKS design decisions that official AWS and Terraform guidance make unavoidable in real deployments. [81]

```
{
  "kind": "aws-platform-spec",
  "version": "v1alpha1",
  "metadata": {
    "name": "payments-prod",
    "environment": "prod",
    "region": "us-east-1"
  },
  "architecture": {
    "compute": {
      "type": "eks",
      "cluster_mode": "standard",
      "kubernetes_version": "1.35",
      "support_policy": "STANDARD",
      "private_cluster": true
    },
    "network": {
      "vpc_cidr": "10.40.0.0/16",
      "az_count": 3,
      "public_ingress": true,
      "egress_mode": "vpc_endpoints_plus_nat",
      "endpoint_services": [
        "ecr.api",
        "ecr.dkr",
        "s3",
        "ec2",
        "elasticloadbalancing",
        "logs",
        "eks",
        "eks-auth",
        "sts"
      ]
```

```
  },
  "compute_profiles": {
    "node_groups": [
      {
        "name": "system-ondemand",
        "capacity_type": "ON_DEMAND",
        "instance_families": ["m7i", "c7i"],
        "min_size": 3,
        "max_size": 12,
        "labels": {"workload-tier": "system"}
      },
      {
        "name": "apps-spot",
        "capacity_type": "SPOT",
        "instance_families": ["m7i", "c7i", "r7i"],
        "min_size": 0,
        "max_size": 50,
        "labels": {"workload-tier": "apps"}
      }
    ],
    "fargate_profiles": [
      {
        "name": "isolated-jobs",
        "selectors": [
          {"namespace": "batch", "labels": {"runtime": "fargate"}}
        ]
      }
    ],
    "autoscaler": {
      "type": "karpenter"
    }
  },
  "identity": {
    "cluster_access_mode": "CAM",
    "workload_identity": "POD_IDENTITY",
    "irsa_enabled_for_fargate": true
  },
  "addons": {
    "core": ["vpc-cni", "coredns", "kube-proxy"],
    "platform": [
      "aws-ebs-csi-driver",
      "aws-efs-csi-driver",
      "amazon-cloudwatch-observability",
      "aws-load-balancer-controller",
      "eks-pod-identity-agent"
    ],
    "cni_mode": "aws-vpc-cni"
  },
```

```
    "storage": {
      "default_class": "ebs-gp3",
      "shared_filesystem": "efs",
      "high_performance_filesystem": null,
      "portable_storage": null
    },
    "edge": {
      "ingress": "alb",
      "api_front_door": "none",
      "cdn": "cloudfront",
      "dns_provider": "route53"
    },
    "security": {
      "pod_security_standard": "restricted",
      "enable_control_plane_logs": [
        "api",
        "audit",
        "authenticator",
        "controllerManager",
        "scheduler"
      ],
      "kms_mode": "customer_managed",
      "ecr_scanning": "enhanced",
      "ecr_lifecycle_policy": "retain-last-30-prod-tags"
    },
    "observability": {
      "container_insights": true,
      "application_signals": true
    }
  }
}
```

Example YAML spec for an EKS recommendation

```
kind: aws-platform-spec
version: v1alpha1
metadata:
  name: ml-platform-dev
  environment: dev
  region: us-west-2

architecture:
  compute:
    type: eks
    cluster_mode: standard
    kubernetes_version: "1.35"
```

```
  support_policy: STANDARD
  private_cluster: false

network:
  vpc_cidr: 10.50.0.0/16
  az_count: 2
  public_ingress: true
  egress_mode: nat
  endpoint_services: ["ecr.api", "ecr.dkr", "s3", "logs", "sts"]

compute_profiles:
  node_groups:
    - name: general
      capacity_type: ON_DEMAND
      instance_families: ["m7i"]
      min_size: 2
      max_size: 10
    - name: gpu
      capacity_type: ON_DEMAND
      instance_families: ["g6"]
      min_size: 0
      max_size: 4
      taints:
        - key: accelerator
          value: gpu
          effect: NoSchedule
  fargate_profiles: []
  autoscaler:
    type: karpenter

identity:
  cluster_access_mode: CAM
  workload_identity: POD_IDENTITY

addons:
  core: ["vpc-cni", "coredns", "kube-proxy"]
  platform:
    - aws-ebs-csi-driver
    - amazon-cloudwatch-observability
    - aws-load-balancer-controller
    - eks-pod-identity-agent
  cni_mode: aws-vpc-cni

storage:
  default_class: ebs-gp3
  shared_filesystem: null
  high_performance_filesystem: fsx_lustre
  portable_storage: null
```

```
  edge:
    ingress: alb
    api_front_door: none
    cdn: none
    dns_provider: route53

  security:
    pod_security_standard: baseline
    enable_control_plane_logs:
      - api
      - audit
      - authenticator
      - controllerManager
      - scheduler
    kms_mode: aws_owned
    ecr_scanning: basic
    ecr_lifecycle_policy: keep-last-10-images

  observability:
    container_insights: true
    application_signals: false
```

Sample Terraform module manifest list

The manifest below shows a good dependency order for generated Terraform. This ordering is consistent with AWS resource dependencies and with HashiCorp's guidance to keep the root orchestrating child modules rather than embedding all logic in a single flat configuration. [82]

```
modules:
  - name: foundation-network
    source: ./modules/foundation-network
    depends_on: []

  - name: foundation-kms
    source: ./modules/foundation-kms
    depends_on: []

  - name: eks-cluster
    source: ./modules/eks-cluster
    depends_on:
      - foundation-network
      - foundation-kms

  - name: eks-nodegroups
    source: ./modules/eks-nodegroups
```

```
    depends_on:
      - eks-cluster

  - name: eks-fargate-profiles
    source: ./modules/eks-fargate-profiles
    depends_on:
      - eks-cluster

  - name: registry-ecr
    source: ./modules/registry-ecr
    depends_on: []

  - name: eks-addons-core
    source: ./modules/eks-addons-core
    depends_on:
      - eks-cluster
      - eks-nodegroups

  - name: eks-addons-platform
    source: ./modules/eks-addons-platform
    depends_on:
      - eks-addons-core
      - registry-ecr

  - name: data-access
    source: ./modules/data-access
    depends_on:
      - foundation-network
      - eks-addons-platform

  - name: edge-routing
    source: ./modules/edge-routing
    depends_on:
      - foundation-network
      - eks-addons-platform

  - name: observability
    source: ./modules/observability
    depends_on:
      - eks-cluster
      - eks-addons-platform
```

Recommended Terraform root conventions

For generated Terraform, use a **single root module per environment** with:

```
terraform {
  required_version = "~> 1.9"

  backend "s3" {
    bucket       = "example-tf-state"
    key          = "platforms/payments-prod/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true
  }

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.40"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.38"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 3.0"
    }
  }
}
```

That pattern follows current Terraform guidance: the root module owns provider configurations and backend configuration, child modules declare their provider requirements, and the dependency lock file should be committed to version control. [83]

A final product recommendation follows from all of the above: if you add EKS, do not model it as a one-click synonym for “containers.” Model it as a **premium-complexity platform path** whose recommendation requires explicit user intent and whose Terraform output includes networking, identity, policy, add-ons, ingress, storage, and observability—not just `aws_eks_cluster` and a node group. That is the difference between a syntax generator and a real infrastructure decision utility. [84]

---

[1] [4] [12] [15] [29] [84] https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html
https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

[2] [16] [22] https://docs.aws.amazon.com/cli/latest/reference/eks/
https://docs.aws.amazon.com/cli/latest/reference/eks/

[3] [26] [72] https://developer.hashicorp.com/terraform/tutorials/modules/pattern-module-creation
https://developer.hashicorp.com/terraform/tutorials/modules/pattern-module-creation

[5] [23] [57] [63] https://aws.amazon.com/eks/pricing/
https://aws.amazon.com/eks/pricing/

6 https://docs.aws.amazon.com/eks/latest/best-practices/multi-account-strategy.html
https://docs.aws.amazon.com/eks/latest/best-practices/multi-account-strategy.html

7 https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-to-cloudfront-distribution.html
https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-to-cloudfront-distribution.html

8 39 64 78 https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html
https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html

9 51 79 https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html
https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html

10 46 59 https://docs.aws.amazon.com/eks/latest/best-practices/security.html
https://docs.aws.amazon.com/eks/latest/best-practices/security.html

11 13 20 24 https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

14 17 https://docs.aws.amazon.com/eks/latest/best-practices/tenant-isolation.html
https://docs.aws.amazon.com/eks/latest/best-practices/tenant-isolation.html

18 33 41 45 77 https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html
https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html

19 43 https://docs.aws.amazon.com/eks/latest/userguide/fargate.html
https://docs.aws.amazon.com/eks/latest/userguide/fargate.html

21 https://aws.amazon.com/lambda/pricing/
https://aws.amazon.com/lambda/pricing/

25 27 28 73 https://docs.aws.amazon.com/eks/latest/userguide/network-reqs.html
https://docs.aws.amazon.com/eks/latest/userguide/network-reqs.html

30 76 https://docs.aws.amazon.com/eks/latest/best-practices/cluster-access-management.html
https://docs.aws.amazon.com/eks/latest/best-practices/cluster-access-management.html

31 https://docs.aws.amazon.com/eks/latest/userguide/enable-iam-roles-for-service-accounts.html
https://docs.aws.amazon.com/eks/latest/userguide/enable-iam-roles-for-service-accounts.html

32 47 https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html
https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html

34 74 https://docs.aws.amazon.com/eks/latest/userguide/fargate-profile.html
https://docs.aws.amazon.com/eks/latest/userguide/fargate-profile.html

35 54 https://docs.aws.amazon.com/eks/latest/userguide/workloads-add-ons-available-eks.html
https://docs.aws.amazon.com/eks/latest/userguide/workloads-add-ons-available-eks.html

36 https://docs.cilium.io/en/stable/installation/cni-chaining-aws-cni.html
https://docs.cilium.io/en/stable/installation/cni-chaining-aws-cni.html

37 42 60 https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html
https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html

38 53 https://docs.aws.amazon.com/AmazonECR/latest/userguide/LifecyclePolicies.html
https://docs.aws.amazon.com/AmazonECR/latest/userguide/LifecyclePolicies.html

40 80 https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Container-Insights-setup-EKS-addon.html
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Container-Insights-setup-EKS-addon.html

44 https://docs.aws.amazon.com/eks/latest/best-practices/vpc-cni.html
https://docs.aws.amazon.com/eks/latest/best-practices/vpc-cni.html

48 https://kubernetes.io/docs/concepts/security/pod-security-standards/
https://kubernetes.io/docs/concepts/security/pod-security-standards/

49 https://docs.aws.amazon.com/eks/latest/best-practices/network-security.html
https://docs.aws.amazon.com/eks/latest/best-practices/network-security.html

50 69 70 81 https://docs.aws.amazon.com/eks/latest/userguide/private-clusters.html
https://docs.aws.amazon.com/eks/latest/userguide/private-clusters.html

52 https://docs.aws.amazon.com/eks/latest/userguide/envelope-encryption.html
https://docs.aws.amazon.com/eks/latest/userguide/envelope-encryption.html

55 https://docs.aws.amazon.com/eks/latest/userguide/configuration-vulnerability-analysis.html
https://docs.aws.amazon.com/eks/latest/userguide/configuration-vulnerability-analysis.html

56 https://docs.aws.amazon.com/eks/latest/userguide/eks-outposts.html
https://docs.aws.amazon.com/eks/latest/userguide/eks-outposts.html

58 https://aws.amazon.com/ecs/pricing/
https://aws.amazon.com/ecs/pricing/

61 https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html
https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html

62 https://aws.amazon.com/fargate/pricing/
https://aws.amazon.com/fargate/pricing/

65 https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-develop-integrations-private.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-develop-integrations-private.html

66 https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/DownloadDistS3AndCustomOrigins.html
https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/DownloadDistS3AndCustomOrigins.html

67 https://docs.aws.amazon.com/eks/latest/best-practices/sgpp.html
https://docs.aws.amazon.com/eks/latest/best-practices/sgpp.html

68 https://docs.aws.amazon.com/vpc/latest/privatelink/gateway-endpoints.html
https://docs.aws.amazon.com/vpc/latest/privatelink/gateway-endpoints.html

71 82 83 https://developer.hashicorp.com/terraform/language/modules/develop/providers
https://developer.hashicorp.com/terraform/language/modules/develop/providers

75 https://docs.aws.amazon.com/eks/latest/best-practices/karpenter.html
https://docs.aws.amazon.com/eks/latest/best-practices/karpenter.html